

```

        is_bold = true
        start_block = index + 1
    ## Handle end of bold text
    ## Remember that the last number in a slice is the index
    ## after the last index used.
    if text[index] == "</B>":
        if not is_bold:
            print("Error: Extra Close Bold")
        print("Bold [", start_block, ":", index, "]",
text[start_block:index])
        is_bold = false
        start_block = index + 1

get_bold(poem)

```

with the output being:

```

Bold [ 1 : 4 ] ['Jack', 'and', 'Jill']
Bold [ 11 : 15 ] ['fetch', 'a', 'pail', 'of']
Bold [ 20 : 23 ] ['down', 'and', 'broke']
Bold [ 28 : 30 ] ['Jill', 'came']

```

The `get_bold()` function takes in a list that is broken into words and tokens. The tokens that it looks for are `<B>` which starts the bold text and `</B>` which ends bold text. The function `get_bold()` goes through and searches for the start and end tokens.

The next feature of lists is copying them. If you try something simple like:

```

>>> a = [1, 2, 3]
>> b = a
>> print(b)
[1, 2, 3]
>> b[1] = 10
>> print(b)
[1, 10, 3]
>> print(a)
[1, 10, 3]

```

This probably looks surprising since a modification to `b` resulted in `a` being changed as well. What happened is that the statement `b = a` makes `b` a *reference* to `a`. This means that `b` can be thought of as another name for `a`. Hence any modification to `b` changes `a` as well. However some assignments don't create two names for one list:

```

>>> a = [1, 2, 3]
>> b = a * 2
>> print(a)
[1, 2, 3]
>> print(b)
[1, 2, 3, 1, 2, 3]
>> a[1] = 10
>> print(a)
[1, 10, 3]
>> print(b)
[1, 2, 3, 1, 2, 3]

```

In this case `b` is not a reference to `a` since the expression `a * 2` creates a new list. Then the statement `b = a * 2` gives `b` a reference to `a * 2` rather than a reference to `a`. All assignment operations create a reference. When you pass a list as an argument to a function you create a reference as well. Most of the time you don't have to worry about creating references rather than copies. However when you need to make modifications to one list without changing another name of the list you have to make sure that you have actually created a copy.

There are several ways to make a copy of a list. The simplest that works most of the time is the slice operator since it always makes a new list even if it is a slice of a whole list:

```
>>> a = [1, 2, 3]
>> b = a[:]
>> b[1] = 10
>> print(a)
[1, 2, 3]
>> print(b)
[1, 10, 3]
```

Taking the slice `[:]` creates a new copy of the list. However it only copies the outer list. Any sublist inside is still a reference to the sublist in the original list. Therefore, when the list contains lists, the inner lists have to be copied as well. You could do that manually but Python already contains a module to do it. You use the `deepcopy` function of the `copy` module:

```
>>> import copy
>> a = [[1, 2, 3], [4, 5, 6]]
>> b = a[:]
>> c = copy.deepcopy(a)
>> b[0][1] = 10
>> c[1][1] = 12
>> print(a)
[[1, 10, 3], [4, 5, 6]]
>> print(b)
[[1, 10, 3], [4, 5, 6]]
>> print(c)
[[1, 2, 3], [4, 12, 6]]
```

First of all notice that `a` is a list of lists. Then notice that when `b[0][1] = 10` is run both `a` and `b` are changed, but `c` is not. This happens because the inner arrays are still references when the slice operator is used. However with `deepcopy` `c` was fully copied.

So, should I worry about references every time I use a function or `=`? The good news is that you only have to worry about references when using dictionaries and lists. Numbers and strings create references when assigned but every operation on numbers and strings that modifies them creates a new copy so you can never modify them unexpectedly. You do have to think about references when you are modifying a list or a dictionary.

By now you are probably wondering why are references used at all? The basic reason is speed. It is much faster to make a reference to a thousand element list than to copy all the elements. The other reason is that it allows you to have a function to modify the inputted list or dictionary. Just remember about references if you ever have some weird problem with data being changed when it shouldn't be.

16 Revenge of the Strings

And now presenting a cool trick that can be done with strings:

```
def shout(string):
    for character in string:
        print("Gimme an " + character)
        print("'" + character + "'")

shout("Lose")

def middle(string):
    print("The middle character is:", string[len(string) // 2])

middle("abcdefg")
middle("The Python Programming Language")
middle("Atlanta")
```

And the output is:

```
Gimme an L
'L'
Gimme an o
'o'
Gimme an s
's'
Gimme an e
'e'
The middle character is: d
The middle character is: r
The middle character is: a
```

What these programs demonstrate is that strings are similar to lists in several ways. The `shout()` function shows that `for` loops can be used with strings just as they can be used with lists. The `middle` procedure shows that strings can also use the `len()` function and array indexes and slices. Most list features work on strings as well.

The next feature demonstrates some string specific features:

```
def to_upper(string):
    ## Converts a string to upper case
    upper_case = ""
    for character in string:
        if 'a' <= character <= 'z':
            location = ord(character) - ord('a')
            new_ascii = location + ord('A')
            character = chr(new_ascii)
        upper_case = upper_case + character
    return upper_case

print(to_upper("This is Text"))
```

with the output being:

```
THIS IS TEXT
```

This works because the computer represents the characters of a string as numbers from 0 to 1,114,111. For example 'A' is 65, 'B' is 66 and 's' is 1488. The values are the unicode¹ value. Python has a function called `ord()` (short for ordinal) that returns a character as a number. There is also a corresponding function called `chr()` that converts a number into a character. With this in mind the program should start to be clear. The first detail is the line: `if 'a' <= character <= 'z':` which checks to see if a letter is lower case. If it is then the next lines are used. First it is converted into a location so that `a = 0`, `b = 1`, `c = 2` and so on with the line: `location = ord(character) - ord('a')`. Next the new value is found with `new_ascii = location + ord('A')`. This value is converted back to a character that is now upper case. Note that if you really need the upper case of a letter, you should use `u=var.upper()` which will work with other languages as well.

Now for some interactive typing exercise:

```
>>> # Integer to String
>> 2
2
>> repr(2)
'2'
>> -123
-123
>> repr(-123)
'-123'
>> # String to Integer
>> "23"
'23'
>> int("23")
23
>> "23" * 2
'2323'
>> int("23") * 2
46
>> # Float to String
>> 1.23
1.23
>> repr(1.23)
'1.23'
>> # Float to Integer
>> 1.23
1.23
>> int(1.23)
1
>> int(-1.23)
-1
>> # String to Float
>> float("1.23")
1.23
>> "1.23"
'1.23'
>> float("123")
123.0
```

¹ <https://en.wikipedia.org/wiki/unicode>

If you haven't guessed already the function `repr()` can convert an integer to a string and the function `int()` can convert a string to an integer. The function `float()` can convert a string to a float. The `repr()` function returns a printable representation of something. Here are some examples of this:

```
>>> repr(1)
'1'
>> repr(234.14)
'234.14'
>> repr([4, 42, 10])
'[4, 42, 10]'
```

The `int()` function tries to convert a string (or a float) into an integer. There is also a similar function called `float()` that will convert an integer or a string into a float. Another function that Python has is the `eval()` function. The `eval()` function takes a string and returns data of the type that python thinks it found. For example:

```
>>> v = eval('123')
>> print(v, type(v))
123 <type 'int'>
>> v = eval('645.123')
>> print(v, type(v))
645.123 <type 'float'>
>> v = eval('[1, 2, 3]')
>> print(v, type(v))
[1, 2, 3] <type 'list'>
```

If you use the `eval()` function you should check that it returns the type that you expect.

One useful string function is the `split()` method. Here's an example:

```
>>> "This is a bunch of words".split()
['This', 'is', 'a', 'bunch', 'of', 'words']
>> text = "First batch, second batch, third, fourth"
>> text.split(",")
['First batch', ' second batch', ' third', ' fourth']
```

Notice how `split()` converts a string into a list of strings. The string is split by whitespace by default or by the optional argument (in this case a comma). You can also add another argument that tells `split()` how many times the separator will be used to split the text. For example:

```
>>> list = text.split(",")
>> len(list)
4
>> list[-1]
' fourth'
>> list = text.split(",", 2)
>> len(list)
3
>> list[-1]
' third, fourth'
```

16.0.1 Slicing strings (and lists)

Strings can be cut into pieces — in the same way as it was shown for lists in the previous chapter — by using the *slicing* "operator" `[]`. The slicing operator works in the same way as before: `text[first_index:last_index]` (in very rare cases there can be another colon and a third argument, as in the example shown below).

In order not to get confused by the index numbers, it is easiest to see them as *clipping places*, possibilities to cut a string into parts. Here is an example, which shows the clipping places (in yellow) and their index numbers (red and blue) for a simple text string:

		0		1		2		...		-2		-1			
		↓		↓		↓		↓		↓		↓		↓	
text	"		S		T		R		I		N		G		"
=															
		↑												↑	
		:												:	

Note that the red indexes are counted from the beginning of the string and the blue ones from the end of the string backwards. (Note that there is no blue -0, which could seem to be logical at the end of the string. Because `-0 == 0`, -0 means "beginning of the string" as well.) Now we are ready to use the indexes for slicing operations:

<code>text[1:4]</code>	→	<code>"TRI"</code>
<code>text[:5]</code>	→	<code>"STRIN"</code>
<code>text[:-1]</code>	→	<code>"STRIN"</code>
<code>text[-4:]</code>	→	<code>"RING"</code>
<code>text[2]</code>	→	<code>"R"</code>
<code>text[:]</code>	→	<code>"STRING"</code>
<code>text[::-1]</code>	→	<code>"GNIRTS"</code>

`text[1:4]` gives us all of the `text` string between clipping places 1 and 4, `"TRI"`. If you omit one of the `[first_index:last_index]` arguments, you get the beginning or end of the string as default: `text[:5]` gives `"STRIN"`. For both `first_index` and `last_index` we can use both the red and the blue numbering schema: `text[:-1]` gives the same as `text[:5]`, because the index -1 is at the same place as 5 in this case. If we do not use an argument containing a colon, the number is treated in a different way: `text[2]` gives us one character following the second clipping point, `"R"`. The special slicing operation `text[:]` means "from the beginning to the end" and produces a copy of the entire string (or list, as shown in the previous chapter).

Last but not least, the slicing operation can have a second colon and a third argument, which is interpreted as the "step size": `text[::-1]` is `text` from beginning to the end, with a step size of -1. -1 means "every character, but in the other direction". `"STRING"` backwards is `"GNIRTS"` (test a step length of 2, if you have not got the point here).

All these slicing operations work with lists as well. In that sense strings are just a special case of lists, where the list elements are single characters. Just remember the concept of *clipping places*, and the indexes for slicing things will get a lot less confusing.

16.0.2 Examples

```
# This program requires an excellent understanding of decimal numbers.
def to_string(in_int):
    """Converts an integer to a string"""
    out_str = ""
    prefix = ""
    if in_int < 0:
        prefix = "-"
        in_int = -in_int
    while in_int // 10 != 0:
        out_str = str(in_int % 10) + out_str
        in_int = in_int // 10
    out_str = str(in_int % 10) + out_str
    return prefix + out_str

def to_int(in_str):
    """Converts a string to an integer"""
    out_num = 0
    if in_str[0] == "-":
        multiplier = -1
        in_str = in_str[1:]
    else:
        multiplier = 1
    for c in in_str:
        out_num = out_num * 10 + int(c)
    return out_num * multiplier

print(to_string(2))
print(to_string(23445))
print(to_string(-23445))
print(to_int("14234"))
print(to_int("12345"))
print(to_int("-3512"))
```

The output is:

```
2
23445
-23445
14234
12345
-3512
```


17 File IO

17.0.1 File I/O

Here is a simple example of file I/O (input/output):

```
# Write a file
with open("test.txt", "wt") as out_file:
    out_file.write("This Text is going to out file\nLook at it and see!")

# Read a file
with open("test.txt", "rt") as in_file:
    text = in_file.read()

print(text)
```

The output and the contents of the file `test.txt` are:

```
This Text is going to out file
Look at it and see!
```

Notice that it wrote a file called `test.txt` in the directory that you ran the program from. The `\n` in the string tells Python to put a *newline* where it is.

An overview of file I/O is:

- Get a file object with the `open` function
- Read or write to the file object (depending on how it was opened)
- If you did not use `with` to open the file, you'd have to close it manually

The first step is to get a file object. The way to do this is to use the `open` function. The format is `file_object = open(filename, mode)` where `file_object` is the variable to put the file object, `filename` is a string with the filename, and `mode` is `"rt"` to read a file as *text* or `"wt"` to write a file as *text* (and a few others we will skip here). Next the file objects functions can be called. The two most common functions are `read` and `write`. The `write` function adds a string to the end of the file. The `read` function reads the next thing in the file and returns it as a string. If no argument is given it will return the whole file (as done in the example).

Now here is a new version of the phone numbers program that we made earlier:

```
def print_numbers(numbers):
    print("Telephone Numbers:")
    for k, v in numbers.items():
        print("Name:", k, "\tNumber:", v)
    print()

def add_number(numbers, name, number):
    numbers[name] = number
```

```
def lookup_number(numbers, name):
    if name in numbers:
        return "The number is " + numbers[name]
    else:
        return name + " was not found"

def remove_number(numbers, name):
    if name in numbers:
        del numbers[name]
    else:
        print(name, " was not found")

def load_numbers(numbers, filename):
    in_file = open(filename, "rt")
    while True:
        in_line = in_file.readline()
        if not in_line:
            break
        in_line = in_line[:-1]
        name, number = in_line.split(",")
        numbers[name] = number
    in_file.close()

def save_numbers(numbers, filename):
    out_file = open(filename, "wt")
    for k, v in numbers.items():
        out_file.write(k + "," + v + "\n")
    out_file.close()

def print_menu():
    print('1. Print Phone Numbers')
    print('2. Add a Phone Number')
    print('3. Remove a Phone Number')
    print('4. Lookup a Phone Number')
    print('5. Load numbers')
    print('6. Save numbers')
    print('7. Quit')
    print()

phone_list = {}
menu_choice = 0
print_menu()
while True:
    menu_choice = int(input("Type in a number (1-7): "))
    if menu_choice == 1:
        print_numbers(phone_list)
    elif menu_choice == 2:
        print("Add Name and Number")
        name = input("Name: ")
        phone = input("Number: ")
        add_number(phone_list, name, phone)
    elif menu_choice == 3:
        print("Remove Name and Number")
        name = input("Name: ")
        remove_number(phone_list, name)
    elif menu_choice == 4:
        print("Lookup Number")
        name = input("Name: ")
        print(lookup_number(phone_list, name))
    elif menu_choice == 5:
        filename = input("Filename to load: ")
        load_numbers(phone_list, filename)
    elif menu_choice == 6:
        filename = input("Filename to save: ")
        save_numbers(phone_list, filename)
    elif menu_choice == 7:
```

```

        break
    else:
        print_menu()

print("Goodbye")

```

Notice that it now includes saving and loading files. Here is some output of my running it twice:

```

1. Print Phone Numbers
2. Add a Phone Number
3. Remove a Phone Number
4. Lookup a Phone Number
5. Load numbers
6. Save numbers
7. Quit

Type in a number (1-7): 2
Add Name and Number
Name: Jill
Number: 1234
Type in a number (1-7): 2
Add Name and Number
Name: Fred
Number: 4321
Type in a number (1-7): 1
Telephone Numbers:
Name: Jill    Number: 1234
Name: Fred    Number: 4321

Type in a number (1-7): 6
Filename to save: numbers.txt
Type in a number (1-7): 7
Goodbye

```

```

1. Print Phone Numbers
2. Add a Phone Number
3. Remove a Phone Number
4. Lookup a Phone Number
5. Load numbers
6. Save numbers
7. Quit

Type in a number (1-7): 5
Filename to load: numbers.txt
Type in a number (1-7): 1
Telephone Numbers:
Name: Jill    Number: 1234
Name: Fred    Number: 4321

Type in a number (1-7): 7
Goodbye

```

The new portions of this program are:

```

def load_numbers(numbers, filename):
    in_file = open(filename, "rt")
    while True:
        in_line = in_file.readline()
        if not in_line:
            break

```

```
        in_line = in_line[:-1]
        name, number = in_line.split(",")
        numbers[name] = number
    in_file.close()

def save_numbers(numbers, filename):
    out_file = open(filename, "wt")
    for k, v in numbers.items():
        out_file.write(k + "," + v + "\n")
    out_file.close()
```

First we will look at the save portion of the program. First it creates a file object with the command `open(filename, "wt")`. Next it goes through and creates a line for each of the phone numbers with the command `out_file.write(k + "," + v + "\n")`. This writes out a line that contains the name, a comma, the number and follows it by a newline.

The loading portion is a little more complicated. It starts by getting a file object. Then it uses a `while True:` loop to keep looping until a `break` statement is encountered. Next it gets a line with the line `in_line = in_file.readline()`. The `readline` function will return an empty string when the end of the file is reached. The `if` statement checks for this and `breaks` out of the `while` loop when that happens. Of course if the `readline` function did not return the newline at the end of the line there would be no way to tell if an empty string was an empty line or the end of the file so the newline is left in what `readline` returns. Hence we have to get rid of the newline. The line `in_line = in_line[:-1]` does this for us by dropping the last character. Next the line `name, number = in_line.split(",")` splits the line at the comma into a name and a number. This is then added to the `numbers` dictionary.

17.0.2 Advanced use of .txt files

You might be saying to yourself, "Well I know how to read and write to a textfile, but what if I want to print the file without opening out another program?"

There are a few different ways to accomplish this. The easiest way does open another program, but everything is taken care of in the Python code, and doesn't require the user to specify a file to be printed. This method involves invoking the subprocess of another program.

Remember the file we wrote output to in the above program? Let's use that file. Keep in mind, in order to prevent some errors, this program uses concepts from the Next chapter. Please feel free to revisit this example after the next chapter.

```
import subprocess
def main():
    try:
        print("This small program invokes the print function in the Notepad
        application")
        #Lets print the file we created in the program above
        subprocess.call(['notepad', '/p', 'numbers.txt'])
    except WindowsError:
        print("The called subprocess does not exist, or cannot be called.")

main()
```

The `subprocess.call` takes three arguments. The first argument in the context of this example, should be the name of the program which you would like to invoke the printing subprocess from. The second argument should be the specific subprocess within that program. For simplicity, just understand that in this program, `'/p'` is the subprocess used to access your printer through the specified application. The last argument should be the name of the file you want to send to the printing subprocess. In this case, it is the same file used earlier in this chapter.

17.0.3 Exercises

Now modify the grades program from section Dictionaries¹ so that it uses file I/O to keep a record of the students.

Solution

Now modify the grades program from section Dictionaries² so that it uses file I/O to keep a record of the students.

```
assignments = ['hw ch 1', 'hw ch 2', 'quiz', 'hw ch 3', 'test']
students = { }
```

```
def load_grades(gradesfile):
    inputfile = open(gradesfile, "r")
    grades = [ ]
    while True:
        student_and_grade = inputfile.readline()
        student_and_grade = student_and_grade[:-1]
        if not student_and_grade:
            break
        else:
            studentname, studentgrades = student_and_grade.split(",")
            studentgrades = studentgrades.split(" ")
            students[studentname] = studentgrades
    inputfile.close()
    print("Grades loaded.")
```

```
def save_grades(gradesfile):
    outputfile = open(gradesfile, "w")
    for k, v in students.items():
        outputfile.write(k + ",")
        for x in v:
            outputfile.write(str(x) + " ")
        outputfile.write("\n")
    outputfile.close()
    print("Grades saved.")
```

```
def print_menu():
    print("1. Add student")
    print("2. Remove student")
    print("3. Load grades")
    print("4. Record grade")
    print("5. Print grades")
    print("6. Save grades")
    print("7. Print Menu")
```

¹ Chapter 13 on page 75

² Chapter 13 on page 75

```
print("9. Quit")

def print_all_grades():
    if students:
        keys = sorted(students.keys())
        print('\t', end=' ')
        for x in assignments:
            print(x, '\t', end=' ')
        print()
        for x in keys:
            print(x, '\t', end=' ')
            grades = students[x]
            print_grades(grades)
    else:
        print("There are no grades to print.")

def print_grades(grades):
    for x in grades:
        print(x, '\t', end=' ')
    print()

print_menu()
menu_choice = 0
while menu_choice != 9:
    print()
    menu_choice = int(input("Menu Choice: "))
    if menu_choice == 1:
        name = input("Student to add: ")
        students[name] = [0] * len(assignments)
    elif menu_choice == 2:
        name = input("Student to remove: ")
        if name in students:
            del students[name]
        else:
            print("Student:", name, "not found")
    elif menu_choice == 3:
        gradesfile = input("Load grades from which file? ")
        load_grades(gradesfile)
    elif menu_choice == 4:
        print("Record Grade")
        name = input("Student: ")
        if name in students:
            grades = students[name]
            print("Type in the number of the grade to record")
            print("Type a 0 (zero) to exit")
            for i, x in enumerate(assignments):
                print(i + 1, x, '\t', end=' ')
            print()
            print_grades(grades)
            which = 1234
            while which != -1:
                which = int(input("Change which Grade: "))
                which -= 1
                if 0 <= which < len(grades):
                    grade = input("Grade: ") # Change from float(input()) to
input() to avoid an error when saving
                    grades[which] = grade
                elif which != -1:
                    print("Invalid Grade Number")
            else:
                print("Student not found")
        elif menu_choice == 5:
            print_all_grades()
        elif menu_choice == 6:
            gradesfile = input("Save grades to which file? ")
            save_grades(gradesfile)
```